

Estruturas de Dados

Teste Laboratorial 1 – 17 de Dezembro de 2021

Nome: _____

Número: _____

Indicações gerais.

- Responda na folha da prova. Utilize os espaços disponibilizados e o verso da folha se necessitar de mais espaço.
- Leia as perguntas com atenção. As soluções pedidas têm frequentemente que obedecer a determinadas restrições. Caso essas restrições não sejam cumpridas, a resposta não é valorizada.
- Sempre que declarar referências para tipos genéricos, deve indicar explicitamente o tipo dos respetivos parametros formais. Não é aceitável definir essas referências sem a definição explícita do tipo. Por outras palavras “List l=new ArrayList();” não é aceitável, enquanto que “List<Object> l=new ArrayList<>();” ou “List<Integer> l=new ArrayList<>();”, por exemplo, são aceitáveis.
- Não pode, nem deve, nem precisa de usar *casts* para resolver estes exercícios.

1 – Considere o seguinte código e indique a complexidade em função de N. Apresente uma breve justificação:

```
for(int i=0;i<N;i++){  
    int M=N>1000?1000:N;  
    for(int j=0;j<M;j++)  
        for(int k=0;k<M;k++)  
            soma ++;  
}
```

R:

Estruturas de Dados

Teste Laboratorial 1 – 17 de Dezembro de 2021

Nome: _____

Número: _____

2- Assuma que dispõe de um método `int pesquisa (int m[], int valor)` que efectua uma pesquisa binária. Este método devolve a posição em que o valor procurado se encontra, ou então um valor negativo (-X) caso este não esteja no array indicado. O valor de `abs (X+1)` indica uma posição em que o valor procurado poderia ser inserido para preservar a ordem. **Este método já existe e não precisa de o fazer.** Construa um método que recebe um array `m`, bem como um valor `x` e indica se esse valor aparece três ou mais vezes no array. O array `m` encontra-se ordenado. **O método deve ter desempenho logarítmico.** Soluções que não respeitem esta restrição não são valorizadas.

Exemplo:

```
int m[]={2,4,4,6,6,6,9,20,20,20,20};  
triplicado(m,4); // falso  
triplicado(m,-12); // falso  
triplicado(m,6); // verdadeiro  
triplicado(m,20); // verdadeiro
```

(Responda no verso)

3- Considere um método `removeComuns` que recebe duas listas de elementos . Os valores da segunda lista devem ser comparáveis com os valores da primeira lista. O método remove da segunda lista todos os valores que aparecem mais do que uma vez na primeira lista.

- Construa o protótipo do método. Soluções que não respeitem as restrições indicadas não são valorizadas.
- Construa o corpo da função. **Deve garantir o melhor desempenho possível, considerando que as listas de entrada tanto poderão ser um `ArrayList` como uma `LinkedList`. Indique qual a complexidade do método em ambos os casos.** Soluções que não respeitem estas indicações não serão valorizadas.

Exemplo de funcionamento:

```
List<Integer> f=Arrays.asList(0,3,2,1,3,2);  
List<Integer> g=new LinkedList<Integer>();  
g.add(3); g.add(4); g.add(2); g.add(1);g.add(22)  
removeComuns(f,g); // g passa a ser {4,1,22}
```

(Responda no verso)